GROUP BY + HAVING

...czyli jak z **wielu wierszy** zrobić **jedno podsumowanie** — ściąga na zajęcia (część 2, bez `JOIN` — cały czas jedna tabela)

1 Rozgrzewka: funkcje agregujące

Funkcja agregująca **zjada wiele wierszy i wypluwa jedną liczbę**. Bez `GROUP BY` liczy dla **całej tabeli** — dlatego wynik ma dokładnie 1 wiersz.

`COUNT(*)` — ile wierszy

Liczy **wiersze**, nie zagląda w wartości. `NULL` też się liczy.

```
SELECT COUNT(*) AS IluUczniow
FROM uczniowie;
```

Efekt: jedna liczba, np. `10`.

`COUNT(kolumna)` — ile wypełnionych

Liczy tylko wiersze, w których ta kolumna **nie jest NULL**.

```
SELECT COUNT(*)      AS Wszyscy,
       COUNT(email)  AS MajaMail
FROM uczniowie;
```

Efekt: `10` i `8` — różnica to liczba pustych maili.

`SUM` i `AVG` — suma i średnia

Tylko na liczbach. `NULL` jest **pomijany**, a nie traktowany jak 0.

```
SELECT SUM(srednia) AS Razem,
       AVG(srednia) AS Przecietnie
FROM uczniowie;
```

Ładniej: `ROUND(AVG(srednia), 2)` — dwie cyfry po kropce.

`MIN` i `MAX` — skrajne wartości

Działają też na tekstach (alfabetycznie) i datach (najstarsza / najnowsza).

```
SELECT MIN(srednia) AS Najgorsza,
       MAX(srednia) AS Najlepsza
FROM uczniowie;
```

Pułapka: to **nie** pokaże, *kto* ma tę średnią — `MAX` zwraca samą liczbę.

`COUNT(DISTINCT …)` — ile różnych

Liczy **unikalne** wartości, powtórki liczone raz.

```
SELECT COUNT(DISTINCT id_klasy) AS IleKlas
FROM uczniowie;
```

Efekt: 3 — trzy różne klasy, choć uczniów jest 10.

Agregat + `WHERE`

`WHERE` najpierw wybiera wiersze, agregat liczy dopiero to, co przeszło filtr.

```
SELECT COUNT(*) AS DobrzyUczniowie
FROM uczniowie
WHERE srednia >= 4.0;
```

Czytaj: „policz wiersze, ale tylko te ze średnią od 4.0".

⚠️ Dwie rzeczy, o których trzeba pamiętać

- **Agregaty ignorują `NULL`** — dlatego `AVG` dzieli przez liczbę *wypełnionych* pól, nie przez wszystkie wiersze.
- **Nie wolno mieszać** agregatu ze zwykłą kolumną bez `GROUP BY`: `SELECT imie, COUNT(*) FROM uczniowie;` to bezsens — jedna liczba i 10 imion nie zmieszczą się w jednym wierszu.

2 `GROUP BY` — dzielimy tabelę na kubełki

`GROUP BY kolumna` mówi: **„posypuj wiersze do kubełków według tej kolumny"**. Potem każda funkcja agregująca liczy **osobno w każdym kubełku**, a w wyniku **jeden kubełek = jeden wiersz**.

8 wierszy w tabeli `uczniowie`

imie	id_klasy	rozszerzony	srednia
Bartosz	1	T	3.80
Ewa	1	N	3.20
Hubert	1	N	3.55
Gabriela	2	T	4.80
Anna	3	T	4.35
Celina	3	T	5.10
Filip	3	N	4.05
Jakub	3	T	3.45

Kolumna `rozszerzony` mówi, czy uczeń chodzi na rozszerzenie (T/N). Zaraz pogrupujemy te same 8 wierszy dwa razy: **raz po jednej kolumnie, raz po dwóch**.

Wariant A — `GROUP BY id_klasy` (jedna kolumna)

↓ `GROUP BY id_klasy` ↓

🪣 id_klasy = 1

Bartosz — 3.80

Ewa — 3.20

Hubert — 3.55

```
COUNT(*) = 3
AVG(srednia) = 3.52
```

🪣 id_klasy = 2

Gabriela — 4.80

```
COUNT(*) = 1
AVG(srednia) = 4.80
```

🪣 id_klasy = 3

Anna — 4.35

Celina — 5.10

Filip — 4.05

Jakub — 3.45

```
COUNT(*) = 4
AVG(srednia) = 4.24
```

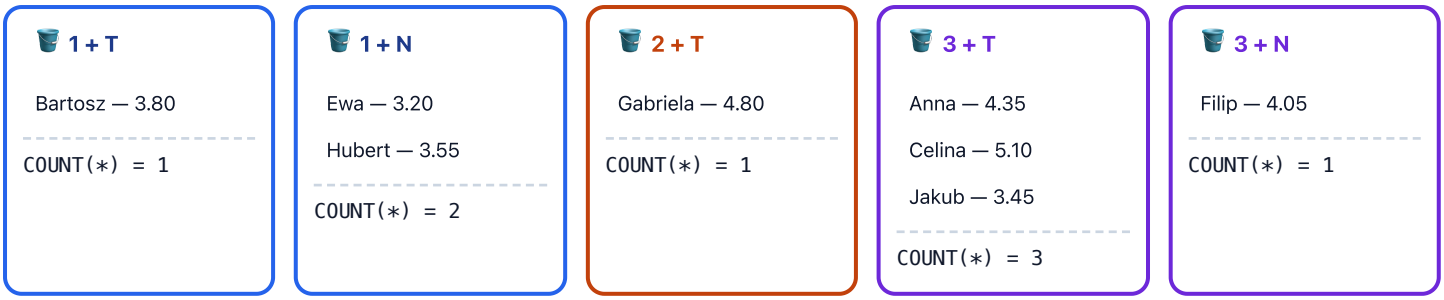
↓ wynik: **3 wiersze**, bo były 3 kubełki ↓

id_klasy	Ilu	SredniaKlasy
1	3	3.52
2	1	4.80
3	4	4.24

Wariant B — `GROUP BY id_klasy, rozszerzony` (dwie kolumny)

Grupować można po **dowolnej liczbie kolumn** — wystarczy wypisać je po przecinku. Kubełek powstaje wtedy dla każdej **kombinacji wartości**, więc grup jest **więcej**, a każda jest **mniejsza**.

↓ `GROUP BY id_klasy, rozszerzony` ↓



↓ wynik: **5 wierszy**, bo było 5 kombinacji ↓

id_klasy	rozszerzony	Ilu
1	T	1
1	N	2
2	T	1
3	T	3
3	N	1

Uwaga: nie ma wiersza 2 + N, bo w danych nie ma ucznia z klasy 2 bez rozszerzenia. Kubełki powstają **tylko dla kombinacji, które faktycznie występują** — SQL nie dopisuje pustych grup.

✅ Złota zasada GROUP BY

W SELECT wolno wpisać **tylko**: kolumny wymienione w GROUP BY **albo** kolumny schowane w funkcji agregującej. Nic innego.

Podstawowy schemat

Kolumna grupująca + agregaty. Prawie zawsze dodajemy ORDER BY.

```
SELECT id_klasy,                -- po tym grupuję
       COUNT(*) AS Ilu          -- liczę w kubełku
FROM uczniowie
GROUP BY id_klasy               -- kubełki po klasach
ORDER BY Ilu DESC;              -- najliczniejsza klasa na górze
```

Efekt: tyle wierszy, ile jest różnych klas.

Kilka agregatów naraz

W jednym zapytaniu można policzyć ile, sumę, średnią i grupy.

```
SELECT id_klasy,
       COUNT(*)              AS Ilu,
       ROUND(AVG(srednia),2) AS SredniaK
       MAX(srednia)          AS Najlepsza
FROM uczniowie
GROUP BY id_klasy;
```

Efekt: jeden wiersz na klasę, cztery kolumny statystyk.

WHERE + GROUP BY

WHERE wyrzuca wiersze **przed** sypaniem do kubełków — więc zmienia liczby w grupach.

```
SELECT id_klasy, COUNT(*) AS Ilu
FROM uczniowie
WHERE srednia IS NOT NULL       -- najpierw filtr wierszy
GROUP BY id_klasy;              -- potem kubełki
```

Uwaga: uczeń bez średniej nie dolicza się do swojej klasy.

Grupowanie po dwóch kolumnach

Kolumny wypisujemy po przecinku. Kubełek = jedna **kor** wartości.

```
SELECT id_klasy, rozszerzony,
       COUNT(*) AS Ilu
FROM uczniowie
GROUP BY id_klasy, rozszerzony
ORDER BY id_klasy, rozszerzony;
```

Efekt: z 3 grup robi się 5 (1+T, 1+N, 2+T, 3+T, 3+N) — **ob** muszą być w SELECT.

Po ilu kolumnach można grupować?

Po dowolnej liczbie — ale każda kolejna **rozdrabnia** grupy.

```
-- 3 kolumny grupujące
SELECT id_klasy, rozszerzony, profil,
       COUNT(*)     AS Ilu,
       AVG(srednia) AS Srednia
FROM uczniowie
GROUP BY id_klasy, rozszerzony, profil;
```

Zasada: im więcej kolumn w `GROUP BY`, tym **więcej wierszy** w wyniku i mniej wierszy w każdej grupie.

Grupowanie a `DISTINCT`

Samo „jakie są wartości" to `DISTINCT`. `GROUP BY` daje **liczby**.

```
SELECT DISTINCT id_klasy FROM uczniowie;

SELECT id_klasy, COUNT(*) FROM uczniowie
GROUP BY id_klasy;
```

Efekt: ta sama lista klas, ale drugie zapytanie mówi też il

Sortowanie po agregacie

W `ORDER BY` **wolno** użyć aliasu z `SELECT` — bo sortowanie jest na końcu.

```
SELECT id_klasy, AVG(srednia) AS SredniaKlasy
FROM uczniowie
GROUP BY id_klasy
ORDER BY SredniaKlasy DESC
LIMIT 1;                          -- najlepsza klasa
```

Zapamiętaj: alias działa w `ORDER BY`, ale **nie** w `WHERE`.

3 HAVING — filtr na grupy

WHERE pyta: „czy ten **wiersz** mnie interesuje?". HAVING pyta: „czy ten **cały kubełek** mnie interesuje?". Dlatego tylko w HAVING można postawić warunek na COUNT, SUM czy AVG.

	WHERE	HAVING
Co filtruje?	pojedyncze **wiersze** tabeli	gotowe **grupy** (kubełki)
Kiedy działa?	**przed** GROUP BY	**po** GROUP BY i po policzeniu agregatów
Wolno tam agregat?	**NIE** — WHERE COUNT(*) > 3 to błąd	**TAK** — HAVING COUNT(*) > 3 ✔
Przykład	WHERE srednia IS NOT NULL odrzuć uczniów bez średniej	HAVING COUNT(*) >= 3 odrzuć klasy mniejsze niż 3 osoby
Kolejność w zapisie	zaraz po FROM	zaraz po GROUP BY

HAVING COUNT(*) — odsiej małe grupy

Klasy, w których jest co najmniej 3 uczniów.

```
SELECT id_klasy, COUNT(*) AS Ilu
FROM uczniowie
GROUP BY id_klasy
HAVING COUNT(*) >= 3;
```

Efekt: grupa z jednym uczniem wypada z wyniku.

HAVING na sumie / średniej

Warunek liczony na całym kubełku, nie na jednym uczniu.

```
SELECT id_klasy, AVG(srednia) AS SredniaKlasy
FROM uczniowie
GROUP BY id_klasy
HAVING AVG(srednia) > 4.0;
```

Czytaj: „pokaż klasy, których **średnia klasowa** przekracza 4.0".

HAVING po aliasie

W MySQL/MariaDB alias z SELECT działa w HAVING — krócej się pisze.

```
SELECT id_klasy, COUNT(*) AS Ilu
FROM uczniowie
GROUP BY id_klasy
HAVING Ilu >= 3;
```

Uwaga: na egzaminie bezpieczniej powtórzyć HAVING COUNT(*) >= 3.

Dwa warunki w HAVING

Można łączyć przez AND / OR, tak jak w WHERE.

```
SELECT id_klasy,
       COUNT(*)     AS Ilu,
       AVG(srednia) AS SredniaKlasy
FROM uczniowie
GROUP BY id_klasy
HAVING COUNT(*) >= 3
   AND AVG(srednia) > 3.5;
```

Efekt: klasa musi być **i** liczna, **i** dobra.

WHERE i HAVING razem

Dwa różne sita: jedno na wiersze, drugie na grupy. Oba w jednym zapytaniu.

```
SELECT id_klasy, COUNT(*) AS Ilu
FROM uczniowie
WHERE srednia IS NOT NULL   -- sito 1: wiersze
GROUP BY id_klasy
HAVING COUNT(*) >= 3;       -- sito 2: grupy
```

Zapamiętaj: to nie konkurenci — normalnie występują razem.

Tak samo na bazie world

Ten sam schemat, inne dane — do zadań na tabeli miasta.

```
SELECT wojewodztwo,
       COUNT(*)                 AS Miasta,
       SUM(liczba_mieszkancow)  AS Ludnosc
FROM miasta
WHERE kod_kraju = 'POL'
GROUP BY wojewodztwo
HAVING COUNT(*) > 2
ORDER BY Ludnosc DESC;
```

Czytaj: „tylko polskie miasta → kubełki po województwach → tylko województwa z ponad 2 miastami".

4 Pełny szkielet zapytania

Numerki po lewej to **kolejność wykonywania** przez serwer — inna niż kolejność pisania! Klauzule piszemy dokładnie w tej kolejności, w jakiej leżą linijki poniżej.

5	`SELECT`	**Które kolumny i agregaty** pokazać? tylko kolumny z GROUP BY albo w funkcji agregującej · tu nadaję aliasy AS
1	`FROM`	**Z której tabeli**? np. `uczniowie`, `miasta`
2	`WHERE` nieobowiązkowe	**Które wiersze** wpuścić do grupowania? filtr WIERSZY — nie wolno tu agregatów ani aliasów
3	`GROUP BY` nieobowiązkowe	**Po czym sypać do kubełków?** tu powstają grupy i liczą się `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`
4	`HAVING` tylko z GROUP BY	**Które grupy** zostają? filtr GRUP — tutaj wolno warunek na agregacie
6	`ORDER BY` nieobowiązkowe	**Jak posortować** gotowe wiersze? tu alias z SELECT już działa · DESC = malejąco
7	`LIMIT` nieobowiązkowe	**Ile wierszy** zostawić? np. `LIMIT 5` — pięć najlepszych grup
	`;`	**Koniec zapytania.** średnik zawsze na samym końcu

⚠️ Najczęstsze błędy

- `WHERE COUNT(*) > 3` → błąd `#1111 Invalid use of group function`. Warunek na agregacie zawsze idzie do `HAVING`.
- `SELECT imie, COUNT(*) … GROUP BY id_klasy` → `imie` nie jest ani w `GROUP BY`, ani w agregacie. MySQL pokaże losowe imię z grupy albo błąd — **nie rób tego**.
- Grupujesz po dwóch kolumnach, a w `SELECT` wpisujesz tylko jedną — wynik jest wtedy nieczytelny (nie wiadomo, której kombinacji dotyczy liczba). Zasada: **wszystkie kolumny z `GROUP BY` pokaż też w `SELECT`**.
- `HAVING` **bez** `GROUP BY` — działa, ale filtruje całą tabelę jako jedną grupę. Prawie zawsze to pomyłka zamiast `WHERE`.
- Zła kolejność: `GROUP BY` **przed** `WHERE` albo `ORDER BY` przed `HAVING` → czerwony błąd składni.
- `AVG(srednia)` na kolumnie z `NULL` — pamiętaj, że puste pola są pomijane, więc mianownik jest mniejszy, niż myślisz.
- Brak `ORDER BY` przy `LIMIT` — „pierwsze 5 grup" bez sortowania to loteria.

Tu się zatrzymujemy.

Łączenie dwóch tabel (`JOIN`) — na kolejnych lekcjach.

5 Klatka po klatce: `GROUP BY` i `HAVING` w akcji

Ta sama tabela `uczniowie` (10 wierszy) co w pierwszej ściądze. Klikaj **Dalej →** i patrz, jak wiersze zsypują się do kubełków, jak liczą się agregaty i dlaczego najlepsza uczennica wypada z wyniku. (Możesz też używać strzałek ← → na klawiaturze.)

```
SELECT id_klasy,
       COUNT(*)              AS Ilu,
       ROUND(AVG(srednia),2) AS SredniaKlasy
FROM uczniowie
WHERE srednia IS NOT NULL
GROUP BY id_klasy
HAVING COUNT(*) >= 3
ORDER BY SredniaKlasy DESC;
```

wiersze: **10** grupy: **–** krok 0 / 9

Serwer dostaje zapytanie

Tabela **uczniowie**: 10 wierszy. Zapytanie ma aż 6 klauzul, ale serwer wykona je **w swojej kolejności** — nie od góry do dołu. Tabela kubełków jest jeszcze pusta.

Tabela `uczniowie` — dane wejściowe

id	imie	nazwisko	id_klasy	rozszerzony	srednia
1	Anna	Kowalska	3	T	4.35
2	Bartosz	Nowak	1	T	3.80
3	Celina	Wójcik	3	T	5.10
4	Damian	Zieliński	2	N	*NULL*
5	Ewa	Lewandowska	1	N	3.20
6	Filip	Szymański	3	N	4.05
7	Gabriela	Dąbrowska	2	T	4.80
8	Hubert	Kamiński	1	N	3.55
9	Iwona	Mazur	2	T	*NULL*
10	Jakub	Piotrowski	3	T	3.45

Kolejność **wykonywania**: `FROM` → `WHERE` → `GROUP BY` → agregaty → `HAVING` → `SELECT` → `ORDER BY`.

6 Jedna kolumna czy dwie? Stepper porównawczy

Te same 8 wierszy (już po `WHERE`) pogrupowane dwa razy: najpierw po **jednej** kolumnie, potem po **dwóch**. Zobacz, jak z 3 kubełków robi się 5 i dlaczego jedna kombinacja w ogóle się nie pojawia.

```
-- 8 uczniów, którzy przeszli filtr
-- WHERE srednia IS NOT NULL
-- pytanie: po czym grupujemy?
```

wiersze: **8** grupy: **–** krok 0 / 5

Te same dane, dwa sposoby grupowania

Po lewej 8 wierszy. Kolumna **id_klasy** ma 3 różne wartości, kolumna **rozszerzony** — 2 (T i N). Od tego, którą (albo które) wpiszemy do `GROUP BY`, zależy liczba wierszy w wyniku.

Uczniowie — 8 wierszy po filtrze

imie	nazwisko	srednia	id_klasy	rozszerzony
Bartosz	Nowak	3.80	1	T
Ewa	Lewandowska	3.20	1	N
Hubert	Kamiński	3.55	1	N
Gabriela	Dąbrowska	4.80	2	T
Anna	Kowalska	4.35	3	T
Celina	Wójcik	5.10	3	T
Jakub	Piotrowski	3.45	3	T
Filip	Szymański	4.05	3	N

Kubełki — jeden wiersz na grupę

?
tu pojawią się kubełki

Liczba wierszy w wyniku = liczba grup. Każda dodatkowa kolumna w `GROUP BY` zwiększa tę liczbę.